

# An Adaptive Framework for Distributed CPU Training: Minimizing Latency and Ensuring Robustness in Unreliable WAN Environments

## Architectural Blueprint for Distributed CPU Training

The transition from Andrej Karpathy's `nanochat`, which utilizes a single-node, multi-GPU architecture via PyTorch's `torchrun` utility, to a distributed, multi-machine CPU-based system necessitates a fundamental re-architecting of its core components [42](#) [92](#). The primary objectives—minimizing training time and ensuring robustness over unstable Wide Area Networks (WANs)—dictate a design centered on efficient communication and resilience, diverging significantly from the high-bandwidth, low-latency environment of multi-GPU setups. The foundational choice for such a distributed system lies in the selection of a communication backend, which dictates the entire framework's compatibility with CPU hardware and heterogeneous network conditions. For all-CPU training, PyTorch's native support directs developers towards the Gloo backend [23](#). Unlike the NVIDIA Collective Communications Library (NCCL), which is highly optimized for GPU interconnects, Gloo is specifically designed for CPU-centric messaging and operates over standard TCP/IP networks, making it the necessary and logical starting point for building a LAN and WAN-capable distributed training system [23](#) [82](#). However, selecting Gloo is merely the first step; its default performance over high-latency, low-bandwidth WAN links is often poor, presenting a significant challenge that cannot be overlooked [77](#) [82](#). This inherent limitation underscores the necessity of layering advanced communication optimization strategies directly onto the chosen backend to achieve the goal of minimizing training time.

The user-specified hybrid parallelism approach, combining data and model parallelism, provides the strategic framework for scaling beyond the memory and computational limits of a single machine [83](#). Data Parallelism (DP) involves creating replicas of the entire model on different machines, with each replica processing a distinct subset of the training batch [46](#). After computation, gradients from all replicas are aggregated (typically via an all-reduce operation) to ensure all models are updated synchronously with the

same global parameters. While DP effectively increases throughput by distributing the workload, it does not solve the problem of a model being too large to fit into the memory of a single machine [46](#). This is where Model Parallelism (MP) becomes essential. MP partitions the model itself across multiple devices. For the Transformer-based architecture of `nanochat`, several MP strategies are applicable. Layer-wise Partitioning is a straightforward method where different layers or groups of layers of the Transformer are assigned to different machines. A more advanced technique is Tensor Parallelism (TP), which splits the weights of individual operations, such as attention heads or linear layers, across multiple GPUs or CPUs [8](#) [37](#). This allows for the training of models with extremely wide layers but introduces significant intra-layer communication complexity to synchronize intermediate results [7](#). Another form, Pipeline Parallelism (PP), partitions the model into sequential stages that execute on different devices, similar to an assembly line [79](#). While PP can handle very deep models, it suffers from inefficiencies known as "bubble time," where some devices may be idle while waiting for data from previous stages, requiring sophisticated scheduling algorithms to maximize hardware utilization [45](#).

The literature highlights a critical caveat regarding conventional hybrid parallelism designs: their performance can be severely degraded in environments with low communication bandwidth between servers [77](#). This finding is directly relevant to the WAN deployment scenario and reinforces the argument that simply implementing DP and MP using the Gloo backend is insufficient. The communication overhead generated by synchronizing millions or billions of parameters will inevitably become the dominant factor in training time, overshadowing any gains from distributed computation [30](#). Therefore, the architectural blueprint must proactively address this communication bottleneck. This requires designing the system not just around parallelism, but around communication efficiency. The fork of `nanochat` must integrate mechanisms to drastically reduce the volume and frequency of data exchanged during training. This includes adopting techniques like gradient compression and strategies to mitigate the impact of high latency, such as delayed aggregation. Furthermore, the overall system design must incorporate fault tolerance from the ground up. Given the instability of WAN connections, the system must be able to detect failed peers, recover from node crashes, and resume training with minimal data loss, moving beyond simple retry logic to a more resilient architecture inspired by modern distributed training frameworks [2](#) [5](#). The final architecture, therefore, is a multi-layered system built upon the Gloo communication foundation, enhanced with communication-efficient parallelism patterns and integrated fault-tolerant mechanisms, all designed to function reliably under the constraints of a heterogeneous, bandwidth-limited, and potentially unreliable network infrastructure.

| Component             | Single-Node Multi-GPU ( <b>nanochat</b> )                 | Proposed Distributed CPU Architecture                                    |
|-----------------------|-----------------------------------------------------------|--------------------------------------------------------------------------|
| Primary Tool          | <code>torchrun</code> 42                                  | Custom implementation using PyTorch's RPC and Gloo backend 23 52         |
| Hardware Focus        | Multiple GPUs per node 42                                 | Multiple CPUs across multiple nodes 62                                   |
| Communication Backend | NCCL (assumed for multi-GPU)                              | GLOO 23                                                                  |
| Parallelism Strategy  | Primarily Model/Data Parallelism within a single node 42  | Hybrid Parallelism (Data + Model) across nodes 77 83                     |
| Network Environment   | High-bandwidth, Low-latency (Intra-node/Inter-GPU)        | LAN/WAN (Variable, often Low-bandwidth, High-latency) 31                 |
| Robustness Mechanism  | Basic fault handling via <code>torchrun</code> logging 16 | Proactive checkpointing, peer monitoring, and adaptive recovery 27 47 48 |

This table illustrates the comprehensive nature of the architectural shift. It moves from a tightly controlled, high-performance computing environment to a loosely coupled, distributed system where communication and reliability are paramount design considerations rather than afterthoughts. The success of the fork hinges on the careful integration of these new components to create a system that is both scalable and resilient.

## Optimizing for Speed: Mitigating the Communication Bottleneck

The primary objective of minimizing training time in a distributed CPU environment, especially over a WAN, is almost entirely dependent on mitigating the communication bottleneck. In a typical distributed training setup, the total time per iteration is the sum of computation time and communication time. While computation time scales favorably with more CPU cores and nodes, communication time does not, and in fact, can become the dominant limiting factor 30 . Research has shown that in hybrid parallelism scenarios, communication can consume as much as 55% of the total GPU time, a figure that would be even more pronounced in a CPU-only setting where raw computational power is lower 30 . In WAN environments characterized by high latency and limited bandwidth, this imbalance is exacerbated, leading to significant throughput degradation where workers spend more time waiting for messages than performing useful computation 31 . To counteract this, the fork of **nanochat** must implement advanced communication reduction techniques, primarily focusing on gradient compression and latency-hiding strategies like delayed aggregation.

Gradient compression is a powerful technique designed to reduce the volume of data that needs to be transmitted between worker nodes during each training step [33](#). Instead of sending the full, high-dimensional gradient vector—which can contain millions or billions of elements—the system transmits only a compressed representation. Two main categories of gradient compression are prevalent in research: quantization and sparsification. Quantization reduces the numerical precision of the gradient values, for instance, by converting them from 32-bit floating-point numbers to 16-bit or even lower, thereby reducing their size by half or more [33](#). Sparsification, on the other hand, aims to transmit only a small fraction of the most "important" gradient elements, typically those with the largest absolute values. Deep Gradient Compression (DGC) is a prominent example of this approach, where elements below a certain threshold are discarded [12](#). To prevent information loss and maintain model convergence, the discarded gradient values are accumulated locally at each node and added back into the gradient vector in subsequent iterations through an error feedback mechanism [14](#). This process ensures that no information is permanently lost, though it introduces a form of noise into the training process. Studies have found that DGC can reduce the amount of gradient data exchanged by up to 99.9%, offering a dramatic reduction in communication overhead [12](#).

While effective, aggressive gradient compression carries the risk of degrading model convergence due to the introduced noise and potential bias [14](#). To stabilize training, sophisticated correction mechanisms are required. One such mechanism is momentum correction, where instead of accumulating raw gradients, the optimizer accumulates the momentum term itself. This helps preserve the correct discounting factor for past gradients, counteracting the effects of staleness caused by asynchronous updates [14](#). Additionally, local gradient clipping is often applied before aggregation to prevent gradient explosion, which can be amplified by the accumulation of local errors [14](#). Another strategy to manage the initial impact of compression is a gradual warmup phase, where the sparsity level is slowly increased over the first few epochs, allowing the model weights to stabilize before aggressive compression is fully engaged [14](#). These techniques demonstrate that while gradient compression is a potent tool for saving bandwidth, its implementation requires careful management to avoid compromising the final quality of the trained model.

Complementing compression is the strategy of delayed aggregation, also known as using staleness, to combat the effects of high network latency [11](#). In a standard synchronous training loop, every worker must wait for all other workers to complete their forward and backward passes before a global parameter update can occur. On a WAN, this synchronization barrier can introduce significant idle time. Delayed aggregation allows workers to proceed with several local computation steps using slightly outdated (stale)

global parameters before synchronizing with the server or other workers [31](#). This pipelines computation and communication, effectively hiding the round-trip time of the network and keeping the CPUs busier. However, this approach introduces another set of challenges related to model divergence due to the use of stale gradients. Critically, research has revealed a complex and negative interaction between staleness and gradient compression. One study explicitly states that "increasing staleness exponentially amplifies the negative effect of gradient compression on model convergence" [31](#). This finding highlights a fundamental three-way trade-off: increasing either compression ratio or staleness can speed up training, but doing so simultaneously harms convergence, making it impossible to find a single static configuration that is optimal under all conditions. This is the central dilemma for a distributed training system operating over an unpredictable WAN. A static strategy might work well initially but fail catastrophically if network conditions suddenly degrade. The state-of-the-art solution to this problem is the development of adaptive optimizers. The DeCo-SGD algorithm, for example, is designed to jointly determine the optimal gradient compression ratio and the number of delayed aggregation steps in real-time, based on continuous measurements of the network's current bandwidth and latency [11](#) [31](#). By dynamically adjusting these hyperparameters, such an optimizer can navigate the complex trade-off space to maximize the end-to-end training speed without sacrificing model accuracy, representing the most promising path toward achieving the objective of minimizing training time in an unstable environment.

## Ensuring Robustness: Designing for Unreliable WAN Environments

The second core objective—to ensure robustness over unstable WAN connections—is a multifaceted challenge that extends beyond simple network error handling. Fault tolerance in distributed systems refers to the ability of a system to continue operating without interruption despite encountering failures, whether they are transient network issues like packet loss or permanent events like node crashes [5](#). In the context of distributed training, an unreliable WAN introduces two primary failure modes: intermittent connectivity leading to dropped packets or high-latency responses, and complete node unavailability [85](#). Building a robust system requires a proactive design that anticipates these failures and incorporates mechanisms for detection, recovery, and continuity. The literature suggests that a multi-layered resilience strategy, combining periodic checkpointing, proactive resource allocation, and intelligent recovery policies, is the most effective approach.

At the heart of any fault-tolerant distributed training system is the checkpointing mechanism [48](#). Checkpointing involves periodically saving the complete state of the training process—including model weights, optimizer states, and the current iteration number—to durable storage. If a node fails or loses connection, the system can be restored to its last valid state, avoiding the need to restart training from scratch and thus minimizing downtime [90](#). Modern frameworks like Oobleck employ pipeline templates to guarantee fault tolerance by strategically placing checkpoints throughout the training process [2](#) [26](#). For the proposed nanochat fork, a hybrid checkpointing strategy would be advisable. This could involve frequent, lightweight checkpoints saved to disk after every few minibatches, combined with less frequent, full-state snapshots. The system should also incorporate a health-checking mechanism, such as periodic RPC pings or heartbeats between nodes, to detect when a peer has become unresponsive. Upon detecting a failure, a predefined recovery protocol would be triggered, such as attempting to re-establish the connection or, if the node is unrecoverable, re-initializing the failed process from the last successful checkpoint. The Gemini system demonstrates the effectiveness of fast failure recovery by checkpointing to CPU memory, a relevant concept since the target architecture does not use GPUs for this purpose [48](#).

Beyond reactive recovery, a more proactive approach involves resource allocation for resilience. The TrainMover runtime, for instance, leverages standby machines to handle interruptions with minimal downtime and zero memory overhead [47](#). In this model, a pool of idle resources is maintained that can be instantly allocated to replace a failed primary node. While this strategy trades off some resource utilization for higher availability, it can be highly effective in cloud environments where resources can be provisioned on demand. For the nanochat fork, this could translate to maintaining a small cluster of "hot spare" machines that can take over the work of a failed worker, drastically reducing the recovery time compared to restarting a process on a cold machine. This aligns with the broader principle of designing resilient distributed systems that anticipate failure rather than merely reacting to it [6](#).

Furthermore, the design can draw inspiration from decentralized paradigms like Federated Learning (FL). While FL is often discussed for its privacy-preserving properties, its decentralized nature offers valuable lessons for building systems that can tolerate partial connectivity [70](#). In a decentralized FL setup, clients can train locally and update a shared model without needing constant, reliable communication with a central server [15](#). This inherent tolerance to network churn makes it a compelling model for WAN deployments. The Odyssey system takes this a step further by proposing an adaptive fault-tolerant system that intelligently selects the optimal recovery strategy based on the type and severity of the detected failure, moving away from a rigid, one-size-fits-all

recovery plan [27](#). This adaptive approach is highly synergistic with the need for an adaptive optimizer to manage the communication-convergence trade-off. The system could, for example, switch to a more conservative, synchronous training mode with more frequent checkpoints if it detects persistent network instability, trading some speed for greater robustness. Finally, it is important to recognize that machine learning algorithms themselves possess a degree of inherent fault tolerance. Research has shown that distributed learning can achieve a comparable convergence rate even when communication channels are unreliable and drop packets with a certain probability [25](#). The training system can be designed to exploit this property, for instance, by continuing to operate with some workers temporarily offline rather than halting the entire process, thereby improving overall resilience and throughput in the face of inevitable network hiccups.

## Integrating Hybrid Parallelism and Computational Efficiency

Successfully implementing the specified hybrid parallelism strategy within the `nanochat` architecture requires careful consideration of both model partitioning and data distribution, all while maximizing the efficiency of CPU-based computation. The goal is to partition the model and data in a way that balances the computational load across all available machines while minimizing the communication traffic between them. For a relatively compact model like `nanochat`, a practical starting point for Model Parallelism (MP) could be a layer-wise partitioning scheme. This involves assigning contiguous blocks of Transformer layers to different machines. For example, if there are 12 Transformer layers and 4 machines, each machine would host 3 layers. During a forward pass, the output of one machine's layers would be sent as input to the next machine, creating a pipelined execution flow. While simple, this approach can lead to load imbalances if the computational cost of different layers varies, and it can suffer from pipeline bubbles if some machines finish their work faster than others [79](#). A more sophisticated approach would be a form of Tensor Parallelism (TP), where the weights of individual operations, such as multi-head attention or feed-forward networks, are split across multiple machines. This allows for the parallel computation of larger matrices but introduces significant intra-layer communication to gather and scatter partial results, which must be carefully managed to avoid becoming a bottleneck [7](#) [8](#).

Regardless of the chosen MP strategy, it must be paired with Data Parallelism (DP) to fully utilize the available hardware. In a combined DP+MP setup, each machine would hold a replica of the partitioned model. The training dataset would then be split into batches, with each machine processing a different shard of the batch. Each machine would perform a complete forward and backward pass on its assigned data shard, producing a "local" gradient. Subsequently, these local gradients must be aggregated across all machines to compute a global gradient, which is then used to update the model parameters. This synchronization step is a major source of communication and is precisely where the previously discussed optimization techniques—gradient compression and delayed aggregation—become critical. The Chimera framework, for instance, was developed specifically to analyze and fuse communication processes in hybrid parallelism scenarios, highlighting the complexity of managing traffic from different sources [39](#).

Beyond parallelization and communication, maximizing computational efficiency on CPUs is paramount. PyTorch's internal operators, such as those for linear algebra and convolutions, are often multi-threaded and can leverage multiple CPU cores to accelerate computation [81](#). The number of threads used by PyTorch can be manually controlled using the `torch.set_num_threads()` function [76](#) [94](#). It is crucial to configure this setting appropriately; using too many threads can lead to oversubscription, where threads compete for CPU resources and actually slow down performance, while using too few threads leaves CPU capacity underutilized [41](#). The optimal number of threads per process is often equal to the number of physical cores on the machine, but this can vary depending on the number of concurrent processes running on that machine. The system should include logic to automatically detect the available CPU cores and set an appropriate thread count to maximize single-node throughput.

A frequently overlooked but critical component for performance in distributed training is the data loading pipeline. Standard data loaders in ML frameworks are often not designed for distributed execution and can become a severe bottleneck, starving the training processes of data and leaving the expensive CPU computations idle [43](#). To minimize training time, the fork must implement a highly efficient, non-blocking data loader capable of streaming data to multiple processes across multiple nodes simultaneously. This involves pre-fetching data from disk, applying transformations and augmentations in parallel, and feeding it into a queue that the training processes can draw from without blocking. Recent advancements in data loading, such as those demonstrated by IBM Research, focus on creating world-class data loaders to enable high-throughput AI model training, underscoring the importance of this component in the overall system performance [84](#). Without a robust and efficient data loading mechanism, the gains achieved through parallelism and communication optimization will

be nullified by I/O bottlenecks, preventing the system from reaching its full potential for minimizing training time.

## Strategic Synthesis and Development Roadmap

The successful development of a fork of `nanochat` for distributed CPU training across LAN and WAN environments is contingent on moving beyond a simple code modification and embracing the creation of a novel, adaptive distributed training system. The dual goals of minimizing training time and ensuring robustness over unstable networks are deeply intertwined and present a complex optimization problem. A static, one-size-fits-all configuration for parallelism, communication, and fault tolerance will inevitably fail. The analysis reveals that the most effective strategy is to build a system that can dynamically adapt to the prevailing network conditions, balancing communication overhead against model convergence and operational stability. The provided research points decisively toward an architecture that integrates an adaptive optimizer, a resilient communication layer, and efficient computational practices into a cohesive whole.

The foundational layer of this system is PyTorch's **GLOO backend**, selected for its native support for CPU-based distributed training over standard TCP/IP networks [23](#). This serves as the bedrock for all inter-process communication. Upon this foundation, a **hybrid parallelism** strategy must be implemented, beginning with a manageable partitioning scheme like layer-wise model parallelism combined with data parallelism to distribute both the model and the workload [77](#) [83](#). However, the cornerstone of minimizing training time is the active mitigation of the communication bottleneck. This requires the integration of advanced techniques. First, **gradient compression**, specifically methods like Deep Gradient Compression (DGC) with error feedback, is essential for reducing the massive data volumes exchanged in WAN environments [12](#) [14](#). Second, to hide high-latency network delays, the system must employ **delayed aggregation (staleness)** to pipeline computation and communication [11](#) [31](#). The critical insight from the research is that these two techniques have a detrimental interaction; increasing one exacerbates the negative impact of the other on model convergence [31](#). This makes a static configuration suboptimal. Therefore, the most strategic path is to implement an **adaptive optimizer**, inspired by frameworks like DeCo-SGD, that can jointly determine the optimal compression ratio and staleness level in real-time based on live network measurements [11](#). This dynamic balancing act is the key to maximizing training throughput without sacrificing model quality.

For ensuring robustness, a multi-layered fault-tolerance strategy is required. This begins with a **hybrid checkpointing** mechanism, involving frequent, lightweight saves to disk for rapid recovery and periodic full-state snapshots for safety [48](#) [90](#). This must be complemented by a peer-monitoring system using RPC timeouts or heartbeats to detect failed nodes proactively. Drawing from systems like TrainMover, the use of **standby machines** could be explored to enable near-instantaneous failover, although this comes at the cost of reduced resource utilization [47](#). The entire system design should be informed by the observation that ML algorithms exhibit a natural tolerance to noisy or incomplete gradient information, meaning the system can be designed to be inherently resilient to the packet loss and intermittency characteristic of WANs [25](#).

Based on this synthesis, a phased development roadmap is recommended:

1. **Phase 1: Baseline and Communication Setup.** Begin by establishing a functional single-node CPU training version of `nanochat`. The first major task is to refactor the training loop to use PyTorch's Distributed RPC Framework and the GLOO backend for inter-process communication. Implement a basic all-reduce operation to synchronize gradients between two processes on a single machine to validate the communication layer.
2. **Phase 2: Hybrid Parallelism Implementation.** Integrate the hybrid parallelism logic. Start with a simple layer-wise model partitioning scheme and pair it with data parallelism. The goal here is to get a working end-to-end distributed training loop that correctly computes gradients and updates model weights across multiple simulated nodes.
3. **Phase 3: End-to-End Optimization.** Address the communication bottleneck. Implement gradient compression (e.g., DGC) with the necessary error feedback and momentum correction mechanisms. Concurrently, integrate the logic for delayed aggregation. Rigorous benchmarking is crucial at this stage to understand the performance impact of these techniques.
4. **Phase 4: Adaptive Control and Resilience.** Develop the adaptive control layer. Create a "network profiler" that runs during a brief warm-up period to characterize network bandwidth and latency. Use these measurements to drive an adaptive optimizer that tunes the compression ratio and staleness dynamically. Simultaneously, implement the checkpointing and peer-monitoring system to provide fault tolerance.
5. **Phase 5: Final Tuning and Deployment.** Conduct extensive testing in a realistic simulated WAN environment (using tools like Wondershaper or NetEm) to fine-tune all parameters. Optimize CPU multi-threading settings and implement a high-performance data loading pipeline to eliminate I/O bottlenecks [41](#) [43](#).

By following this structured approach, the project can systematically build a robust and high-performance distributed training system, transforming the nanochat repository into a valuable contribution to the field of resource-constrained and resilient machine learning.

---

## Reference

1. A Survey on Fault-tolerance in Distributed Optimization and Machine ... <https://arxiv.org/abs/2106.08545>
2. Resilient Distributed Training of Large Models Using Pipeline ... <https://dl.acm.org/doi/abs/10.1145/3600006.3613152>
3. Fault tolerance in distributed systems using deep learning approaches <https://pmc.ncbi.nlm.nih.gov/articles/PMC11706390/>
4. A Survey on Fault-tolerance in Distributed Optimization and Machine ... <https://www.semanticscholar.org/paper/A-Survey-on-Fault-tolerance-in-Distributed-and-Liu/972be47cdd1b43f2f8245a8e34efcd6d7f251c4f>
5. Fault Tolerance in Distributed Systems: Strategies and Case Studies <https://dev.to/nekto0n/fault-tolerance-in-distributed-systems-strategies-and-case-studies-29d2>
6. (PDF) Designing Resilient Distributed Systems: Fault Tolerance ... [https://www.researchgate.net/publication/389533767\\_Designing\\_Resilient\\_Distributed\\_Systems\\_Fault\\_Tolerance\\_Strategies\\_and\\_Insights](https://www.researchgate.net/publication/389533767_Designing_Resilient_Distributed_Systems_Fault_Tolerance_Strategies_and_Insights)
7. Distributed Hybrid Parallelism for Large Language Models - arXiv <https://arxiv.org/html/2602.09109v1>
8. Distributed inference with vLLM - Red Hat Developer <https://developers.redhat.com/articles/2025/02/06/distributed-inference-with-vllm>
9. Distributed Training of Large Language Models Across Multiple ... <https://www.rohan-paul.com/p/distributed-training-of-large-language>
10. Distributed Training for LLMs <https://www.genaimlinstitute.com/blog/distributed-training-for-llms>
11. Taming Latency and Bandwidth: A Theoretical Framework ... - arXiv <https://arxiv.org/html/2507.17346v2>

12. Reducing the Communication Bandwidth for Distributed Training <https://www.semanticscholar.org/paper/Deep-Gradient-Compression%3A-Reducing-the-Bandwidth-Lin-Han/92495abbac86394cb759bec15a763dbf49a8e590>
13. A Joint Approach to Local Updating and Gradient Compression for ... [https://www.researchgate.net/publication/382079936\\_A\\_Joint\\_Approach\\_to\\_Local\\_Updating\\_and\\_Gradient\\_Compression\\_for\\_Efficient\\_Asynchronous\\_Federated\\_Learning](https://www.researchgate.net/publication/382079936_A_Joint_Approach_to_Local_Updating_and_Gradient_Compression_for_Efficient_Asynchronous_Federated_Learning)
14. ICLR 2018 | Deep Gradient Compression: Reducing the ... - 博客园 <https://www.cnblogs.com/shuo-ouyang/p/12944258.html>
15. [PDF] Fault-Tolerant Decentralized Distributed Asynchronous Federated ... <https://arxiv.org/pdf/2509.02186>
16. Fault-tolerant Distributed Training with torchrun — PyTorch Tutorials ... [https://docs.pytorch.org/tutorials/beginner/ddp\\_series\\_fault\\_tolerance.html](https://docs.pytorch.org/tutorials/beginner/ddp_series_fault_tolerance.html)
17. Fault-Tolerance Performance in WAN. | Download Scientific Diagram [https://www.researchgate.net/figure/Fault-Tolerance-Performance-in-WAN\\_fig2\\_261453343](https://www.researchgate.net/figure/Fault-Tolerance-Performance-in-WAN_fig2_261453343)
18. Fault – tolerance approaches for distributed and cloud computing ... <https://onlinelibrary.wiley.com/doi/10.1002/cpe.8081>
19. In-depth analysis of fault tolerant approaches integrated with load ... <https://link.springer.com/article/10.1007/s12083-024-01798-5>
20. RPC Considered Harmful: Fast Distributed Deep Learning on RDMA <https://arxiv.org/html/1805.08430>
21. Distributed Model Parallel Using Distributed RPC - PyTorch Forums <https://discuss.pytorch.org/t/distributed-model-parallel-using-distributed-rpc/87875>
22. [PDF] Distributed Deep Learning - GRSS-IEEE [https://www.grss-ieee.org/wp-content/uploads/2024/02/Lecture\\_3\\_Distributed\\_Deep\\_Learning.pdf](https://www.grss-ieee.org/wp-content/uploads/2024/02/Lecture_3_Distributed_Deep_Learning.pdf)
23. Distributed communication package - PyTorch documentation <https://docs.pytorch.org/docs/stable/distributed.html>
24. Single-machine, single-GPU: distributed best practices <https://discuss.pytorch.org/t/single-machine-single-gpu-distributed-best-practices/169243>
25. [1810.07766] Distributed Learning over Unreliable Networks - arXiv <https://arxiv.org/abs/1810.07766>
26. Resilient Distributed Training of Large Models <https://micde.umich.edu/project/resilient-distributed-training-of-large-models/>
27. Odyssey: Adaptive Policy Selection for Resilient Distributed Training <https://arxiv.org/html/2508.21613v1>
28. A New Computing Paradigm for GPU-CPU Parallel LLM Inference <https://dl.acm.org/doi/10.1145/3688351.3689164>

29. Simplifying the Training of an LLM Model Using 1024 GPUs - MDPI <https://www.mdpi.com/2076-3417/15/15/8265>
30. LLM Training Parallelism: A Practical Guide to Choosing the Right ... <https://www.linkedin.com/pulse/llm-training-parallelism-practical-guide-choosing-right-deepak-soni-r3exe>
31. DeCo-SGD: Joint Optimization of Delay Staleness and Gradient ... <https://arxiv.org/html/2507.17346v1>
32. Communication Sparsification for Distributed Deep Learning Training <https://dl.acm.org/doi/10.1145/3563038>
33. Communication-efficient distributed learning with Local Immediate ... <https://www.sciencedirect.com/science/article/abs/pii/S0893608025003508>
34. Where do I get a CPU-only version of PyTorch? - Stack Overflow <https://stackoverflow.com/questions/51730880/where-do-i-get-a-cpu-only-version-of-pytorch>
35. FTPipeHD: A Fault-Tolerant Pipeline-Parallel Distributed Training ... <https://arxiv.org/abs/2110.02781>
36. (PDF) Optimizing Latency in Distributed Consensus Protocols: A ... [https://www.researchgate.net/publication/393510690\\_Optimizing\\_Latency\\_in\\_Distributed\\_Consensus\\_Protocols\\_A\\_Framework\\_for\\_Reducing\\_Commit\\_Delays\\_in\\_Fault-Tolerant\\_and\\_Scalable\\_Systems](https://www.researchgate.net/publication/393510690_Optimizing_Latency_in_Distributed_Consensus_Protocols_A_Framework_for_Reducing_Commit_Delays_in_Fault-Tolerant_and_Scalable_Systems)
37. Serving Hybrid LLM Loads with SLO Guarantees Using CPU-GPU ... <https://arxiv.org/html/2603.12831v1>
38. Distributed training of large language models: A survey <https://www.sciencedirect.com/science/article/pii/S2949719125000500>
39. Chimera: Communication Fusion for Hybrid Parallelism in Large ... <https://dl.acm.org/doi/10.1145/3695053.3731025>
40. Total number of processes and threads created using nn.distributed ... <https://discuss.pytorch.org/t/total-number-of-processes-and-threads-created-using-nn-distributed-parallel/71043>
41. What is the recommended number of threads for PyTorch relative to ... <https://stackoverflow.com/questions/76084214/what-is-the-recommended-number-of-threads-for-pytorch-relative-to-available-cpu>
42. Anyone got nanochat training working on the DGX spark? <https://forums.developer.nvidia.com/t/anyone-got-nanochat-training-working-on-the-dgx-spark/348537>
43. [PDF] The Streaming Batch Model for Efficient and Fault-Tolerant ... - arXiv <https://arxiv.org/pdf/2501.12407>

44. Maximizing Parallelism in Distributed Training for Huge Neural ... <https://www.semanticscholar.org/paper/Maximizing-Parallelism-in-Distributed-Training-for-Bian-Xu/d8df456f790381f4ddb388be24a546625bd75ee2>
45. FTPipeHD: A Fault-Tolerant Pipeline-Parallel Distributed Training ... [https://www.researchgate.net/publication/371245861\\_FTPipeHD\\_A\\_Fault-Tolerant\\_Pipeline-Parallel\\_Distributed\\_Training\\_Approach\\_for\\_Heterogeneous\\_Edge\\_Devices](https://www.researchgate.net/publication/371245861_FTPipeHD_A_Fault-Tolerant_Pipeline-Parallel_Distributed_Training_Approach_for_Heterogeneous_Edge_Devices)
46. Distributed Training Design | MindSpore 2.0 documentation [https://www.mindspore.cn/docs/en/r2.0/design/distributed\\_training\\_design.html](https://www.mindspore.cn/docs/en/r2.0/design/distributed_training_design.html)
47. TrainMover: An Interruption-Resilient and Reliable ML Training ... <https://arxiv.org/html/2412.12636v2>
48. GEMINI: Fast Failure Recovery in Distributed Training with In ... <https://dl.acm.org/doi/10.1145/3600006.3613145>
49. Open-source training framework increases the speed of large ... <https://cse.engin.umich.edu/stories/open-source-training-framework-increases-the-speed-of-large-language-model-pre-training-when-failures-arise>
50. How to Set Up Cloud-Based Distributed Training to Fine-Tune an LLM <https://www.intel.com/content/www/us/en/developer/articles/technical/set-up-cloud-based-distributed-training.html>
51. Asteroid: Resource-Efficient Hybrid Pipeline Parallelism for ... - arXiv <https://arxiv.org/html/2408.08015v1>
52. Distributed RPC Framework — PyTorch 2.12 documentation <https://docs.pytorch.org/docs/stable/rpc.html>
53. I Rebuilt Karpathy's NanoChat in JAX. Here's What XLA Gets Right ... <https://dev.to/gde/i-rebuilt-karpathys-nanochat-in-jax-heres-what-xla-gets-right-and-what-it-gets-dead-wrong-4641>
54. karpathy/autoresearch: AI agents running research on single-GPU ... [https://www.linkedin.com/posts/niallm\\_github-karpathyautoresearch-ai-agents-activity-7437230871096573952-GnH9](https://www.linkedin.com/posts/niallm_github-karpathyautoresearch-ai-agents-activity-7437230871096573952-GnH9)
55. 小破本也能炼GPT! 300行代码搞定, GitHub获6k星, 来自特斯拉前AI ... <https://zhuanlan.zhihu.com/p/600586039>
56. A New Open-Source Library for Distributed Training in JAX - LinkedIn [https://www.linkedin.com/posts/aman-swar\\_github-amanswardistjax-a-mini-jax-library-activity-7368219428917805056-b7nN](https://www.linkedin.com/posts/aman-swar_github-amanswardistjax-a-mini-jax-library-activity-7368219428917805056-b7nN)
57. Efficient Training of Large Language Models on Distributed ... - arXiv <https://arxiv.org/html/2407.20018v1>
58. Efficient training of large language models on distributed ... <https://link.springer.com/article/10.1007/s44336-026-00038-z>

59. Get Exclusive ChatGPT in 4 Hours: Karpathy's Old Tricks, 8,000 ... <https://eu.36kr.com/en/p/3508936344689539>
60. Data-Driven Mobility System for Vehicular Communication <https://ieeexplore.ieee.org/iel8/6287639/10820123/11034976.pdf>
61. [PDF] OECD Digital Education Outlook 2026 (EN) [https://www.oecd.org/content/dam/oecd/en/publications/reports/2026/01/oecd-digital-education-outlook-2026\\_940e0dd8/062a7394-en.pdf](https://www.oecd.org/content/dam/oecd/en/publications/reports/2026/01/oecd-digital-education-outlook-2026_940e0dd8/062a7394-en.pdf)
62. Training a small LLM with Keras and commodity hardware - LinkedIn [https://www.linkedin.com/posts/pawanlamba\\_nanogpt-llmfromscratch-llm-activity-7370105058497056768-fs2r](https://www.linkedin.com/posts/pawanlamba_nanogpt-llmfromscratch-llm-activity-7370105058497056768-fs2r)
63. Collective Communication for 100k+ GPUs - arXiv <https://arxiv.org/html/2510.20171v1>
64. Federated Learning-Driven IoT Request Scheduling for Fault ... - MDPI <https://www.mdpi.com/2227-7390/13/13/2198>
65. [PDF] Providing Efficient Fault Tolerance in Distributed ... - UC Berkeley <https://escholarship.org/content/qt0pb7p69t/qt0pb7p69t.pdf?v=lg>
66. Architecting Peer-to-Peer Serverless Distributed Machine Learning ... [https://www.researchgate.net/publication/368843266\\_Architecting\\_Peer-to-Peer\\_Serverless\\_Distributed\\_Machine\\_Learning\\_Training\\_for\\_Improved\\_Fault\\_Tolerance](https://www.researchgate.net/publication/368843266_Architecting_Peer-to-Peer_Serverless_Distributed_Machine_Learning_Training_for_Improved_Fault_Tolerance)
67. distributed high-performance computing methods for accelerating ... [https://www.researchgate.net/publication/384325589\\_DISTRIBUTED\\_HIGH-PERFORMANCE\\_COMPUTING\\_METHODS\\_FOR\\_ACCELERATING\\_DEEP\\_LEARNING\\_TRAINING](https://www.researchgate.net/publication/384325589_DISTRIBUTED_HIGH-PERFORMANCE_COMPUTING_METHODS_FOR_ACCELERATING_DEEP_LEARNING_TRAINING)
68. [PDF] IDENTIFYING AND MITIGATING ERRORS IN GRADI - OpenReview <https://openreview.net/pdf?id=zOgUQM6uLZ>
69. [PDF] Oobleck: Resilient Distributed Training of Large Models Using ... <https://arxiv.org/pdf/2309.08125>
70. Federated Learning for IoT: Applications, Trends, Taxonomy ... <https://ieeexplore.ieee.org/iel8/8782661/10362961/10767353.pdf>
71. Karpathy's Autoresearch Project: AI Coding Agent for ML Experiments [https://www.linkedin.com/posts/max-forsey\\_github-karpathyautoresearch-ai-agents-activity-7437621965080846337-hVCy](https://www.linkedin.com/posts/max-forsey_github-karpathyautoresearch-ai-agents-activity-7437621965080846337-hVCy)
72. Andrej Karpathy Open-Sources AutoResearch for AI Training - TikTok <https://www.tiktok.com/@github.awesome/video/7614524799255055629>
73. Karpathy's Autoresearch Breakthrough: 11% Gains in 700 Iterations [https://www.linkedin.com/posts/kukeshkodess\\_github-karpathyautoresearch-ai-agents-activity-7437337755996737536-rOkO](https://www.linkedin.com/posts/kukeshkodess_github-karpathyautoresearch-ai-agents-activity-7437337755996737536-rOkO)

74. CORAL: Towards Autonomous Multi-Agent Evolution for Open ... <https://arxiv.org/html/2604.01658v1>
75. A Comprehensive Survey of Small Language Models in the Era of ... <https://arxiv.org/html/2411.03350v1>
76. pytorch中神经网络的多线程数设置: torch.set\_num\_threads(N) <https://www.cnblogs.com/xyz/p/15854307.html>
77. Distributed Parallelism Overview | MindSpore master Tutorials <https://www.mindspore.cn/tutorials/en/master/parallel/overview.html>
78. [PDF] Cross-Cluster Auto-Parallelism for Heterogeneous Distributed Training <https://i.cs.hku.hk/~cwu/papers/gcqi-eurosys26.pdf>
79. Pipeline Parallelism — PyTorch 2.12 documentation <https://docs.pytorch.org/docs/stable/distributed.pipelining.html>
80. A major shift is happening in the LLM industry. LLMs just learned to ... [https://www.linkedin.com/posts/vladlarichev\\_a-major-shift-is-happening-in-the-llm-activity-7439246178526892032-poGU](https://www.linkedin.com/posts/vladlarichev_a-major-shift-is-happening-in-the-llm-activity-7439246178526892032-poGU)
81. How to parallelize a training loop over samples of a batch when ... <https://stackoverflow.com/questions/66226135/how-to-parallelize-a-training-loop-over-samples-of-a-batch-when-cpu-is-only-avail>
82. Why GLOO's performance is much worse than NCCL? - distributed <https://discuss.pytorch.org/t/why-gloos-performance-is-much-worse-than-nccl/194008>
83. Distributed training | MindSpore 0.6 Tutorials [https://www.mindspore.cn/tutorial/en/r0.6/advanced\\_use/distributed\\_training\\_tutorials.html](https://www.mindspore.cn/tutorial/en/r0.6/advanced_use/distributed_training_tutorials.html)
84. PyTorch 2024: Training AI models faster than ever - IBM Research <https://research.ibm.com/blog/pytorch-2024-training-models>
85. Convergence of Byzantine-Resilient Gradient Tracking via ... [https://www.researchgate.net/publication/403427304\\_Convergence\\_of\\_Byzantine-Resilient\\_Gradient\\_Tracking\\_via\\_Probabilistic\\_Edge\\_Dropout](https://www.researchgate.net/publication/403427304_Convergence_of_Byzantine-Resilient_Gradient_Tracking_via_Probabilistic_Edge_Dropout)
86. Karpathy's Autonomous AI Research: A Decade of Expertise - LinkedIn [https://www.linkedin.com/posts/aagupta\\_karpathy-buried-the-most-important-part-of-activity-7442819160356872192-XBnw](https://www.linkedin.com/posts/aagupta_karpathy-buried-the-most-important-part-of-activity-7442819160356872192-XBnw)
87. AutoKernel: Autonomous GPU Kernel Optimization via Iterative ... <https://arxiv.org/html/2603.21331v1>
88. Open-source Scalable LLM Training on GPU-based Supercomputers <https://arxiv.org/html/2502.08145>
89. TensorLang Lands Native Training Loops for CUDA Kernels - LinkedIn [https://www.linkedin.com/posts/davidgstevens\\_github-davrotensor-lang-tensorlang-is-activity-7451877905569181696-m7Vm](https://www.linkedin.com/posts/davidgstevens_github-davrotensor-lang-tensorlang-is-activity-7451877905569181696-m7Vm)

90. Ooblock: Resilient Distributed Training of Large Models Using ... [https://www.researchgate.net/publication/374920388\\_Ooblock\\_Resilient\\_Distributed\\_Training\\_of\\_Large\\_Models\\_Using\\_Pipeline\\_Templates](https://www.researchgate.net/publication/374920388_Ooblock_Resilient_Distributed_Training_of_Large_Models_Using_Pipeline_Templates)
91. Train an LLM from Scratch with Karpathy's Nanochat - LinkedIn [https://www.linkedin.com/posts/ronanmcgovern\\_train-an-llm-from-scratch-with-karpathys-activity-7401621307328724992-N1u0](https://www.linkedin.com/posts/ronanmcgovern_train-an-llm-from-scratch-with-karpathys-activity-7401621307328724992-N1u0)
92. 1 Introduction - arXiv <https://arxiv.org/html/2511.13761v1>
93. Efficiently Scale LLM Training Across a Large GPU Cluster with Alpa ... <https://developer.nvidia.com/blog/efficiently-scale-llm-training-across-a-large-gpu-cluster-with-alpa-and-ray/>
94. 【PyTorch】 torch.set\_num\_threads( ) 原创 - CSDN博客 <https://blog.csdn.net/wq6qeg88/article/details/144731955>
95. 9 libraries for parallel & distributed training/inference of deep ... <https://zhuanlan.zhihu.com/p/625249098>